

JAVA PROGRAMMING

Chapter 28: Multithreading in Java

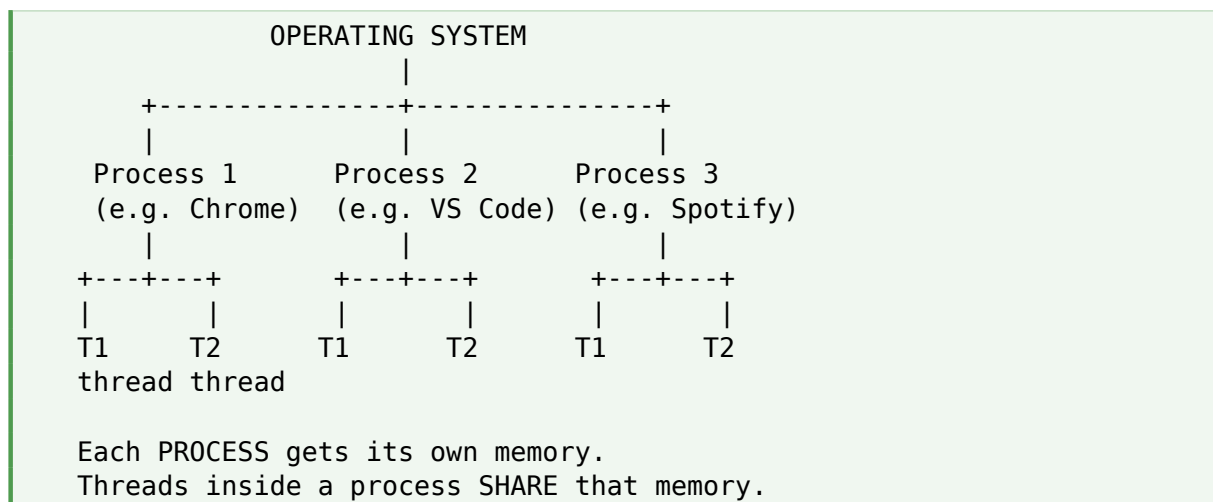
Personal Notes

Up to now, every Java program in these notes has run on a SINGLE thread — one line at a time, top to bottom. But modern computers have multiple CPU cores, and real applications need to do many things at once: handle network requests, animate UIs, process files in parallel, run background tasks.

Multithreading is Java's way of doing more than one thing at the same time. It is one of the most powerful — and most dangerous — features of the language. Get it right, and your code runs faster and stays responsive. Get it wrong, and you create bugs that only happen on Tuesdays at 3 AM.

1. Process vs Thread

Before threads, you need to know what a PROCESS is — and how threads fit in.



- Process — an independent program running on the OS. Has its own memory space.
- Thread — a unit of execution INSIDE a process. Threads share memory with each other.
- Every Java program starts with ONE thread (the "main" thread). You can create more from there.

Why threads, not processes: Threads are lightweight — fast to create, low memory overhead, easy to share data (they share the same heap). Processes are heavy — separate memory, expensive to start, hard to communicate between. For most concurrency inside an app, you want threads.

2. Why Multithreading?

Multithreading exists for two main reasons:

2.1 Responsiveness

UI apps must stay responsive even when doing slow work in the background. If a 5-second file download froze the UI, users would scream. Run the download on a separate thread → UI stays smooth.

2.2 Parallelism (Speed)

A 4-core CPU can run 4 things at the same time. If you have CPU-heavy work that can be split (image processing, sorting big arrays, calculating frame data), multithreading uses ALL the cores instead of just one — potentially 4x faster.

Single-threaded	Multi-threaded
Core 1: ██████████ Core 2: (idle) Core 3: (idle) Core 4: (idle)	Core 1: ██████ Core 2: ██████ Core 3: ██████ Core 4: ██████
12 seconds	3 seconds

Important reality check: Threads don't magically make any program 4x faster. Only INDEPENDENT, CPU-bound work benefits. Database calls or network requests don't get faster with more threads — they're waiting on something else. And work that has to be done in a strict order can't be parallelized at all.

3. Creating Threads

There are 3 main ways to create a thread in Java. Each fits different situations.

3.1 Way 1: Extending the Thread Class

```
class MyThread extends Thread {
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println("MyThread: " + i);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();    // starts a new thread, calls run() in it

        // Main thread continues here in parallel
        for (int i = 0; i < 5; i++) {
            System.out.println("Main: " + i);
        }
    }
}
```

```

    }
}

```

Output is INTERLEAVED — Main and MyThread lines mix together because both threads run in parallel.

Critical: start() vs run() Calling `t.start()` creates a NEW thread and runs the `run()` method inside it — true parallelism. Calling `t.run()` directly just runs the method on the current thread — NO new thread, NO parallelism. This mistake is one of the most common bugs in beginner threading code.

3.2 Way 2: Implementing Runnable (Preferred)

```

class MyTask implements Runnable {
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println("Task: " + i);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Thread t = new Thread(new MyTask());
        t.start();
    }
}

```

Runnable is preferred because Java doesn't allow multiple inheritance for classes. Extending Thread blocks you from extending anything else. Implementing Runnable leaves you free.

3.3 Way 3: Lambda Runnable (Modern, Cleanest)

```

public class Main {
    public static void main(String[] args) {
        Thread t = new Thread(() -> {
            for (int i = 0; i < 5; i++) {
                System.out.println("Lambda thread: " + i);
            }
        });

        t.start();
    }
}

```

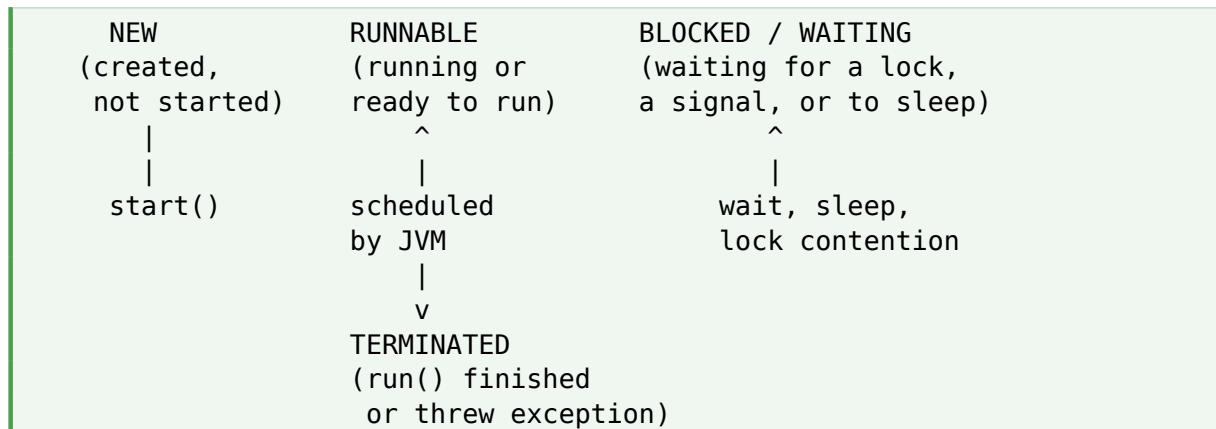
Since Runnable is a functional interface (one abstract method: `run()`), you can pass a lambda directly. This is what modern Java code looks like.

Which One to Use?

Approach	When to use
extends Thread	Avoid — locks you out of other inheritance
implements Runnable	Good for reusable task classes
Lambda Runnable	Best for short, inline tasks — most common in real code

4. Thread Lifecycle

A thread moves through these states during its life:



State	Meaning
NEW	Thread object created but start() not called yet
RUNNABLE	Eligible to run — either running now or waiting for CPU
BLOCKED	Waiting to acquire a lock (synchronized)
WAITING	Waiting indefinitely for another thread (e.g. join(), wait())
TIMED_WAITING	Waiting for a fixed time (e.g. sleep(1000))
TERMINATED	run() has completed (or thrown an exception)

5. Common Thread Methods

5.1 sleep(milliseconds) — Pause a Thread

```
Thread.sleep(2000); // pauses current thread for 2 seconds
```

```
// Must handle InterruptedException (checked exception)
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();    // restore interrupted flag
}
```

sleep() pauses the CURRENT thread (whichever one is calling it). The thread stays in TIMED_WAITING state until the duration is up.

5.2 join() — Wait for Another Thread to Finish

```
Thread worker = new Thread(() -> {
    // do some long task
    System.out.println("Worker done");
});

worker.start();
worker.join();    // main thread waits here until worker finishes
System.out.println("Main continues");
```

join() blocks the calling thread until the target thread finishes. Useful when one thread depends on another's result.

5.3 setName / getName

```
Thread t = new Thread(() -> doWork());
t.setName("DataWorker");
t.start();

System.out.println(t.getName());                // DataWorker
System.out.println(Thread.currentThread().getName()); // "main"
```

5.4 setDaemon — Background Thread

Daemon threads are background helpers that DON'T prevent the JVM from exiting. When all non-daemon threads finish, the JVM kills all daemons and shuts down.

```
Thread t = new Thread(() -> { /* background work */ });
t.setDaemon(true);    // must be called BEFORE start()
t.start();
```

5.5 interrupt() — Politely Ask a Thread to Stop

```
Thread t = new Thread(() -> {
    while (!Thread.currentThread().isInterrupted()) {
        // do work
    }
    System.out.println("Stopping cleanly");
});
```

```
});

t.start();
Thread.sleep(2000);
t.interrupt(); // request the thread to stop
```

Why interrupt(), not stop(): Thread.stop() was a method that killed threads instantly — and is now DEPRECATED because it could leave shared data in a corrupted state. interrupt() is the modern, safe way: it sets a flag, and the thread checks that flag and exits gracefully.

6. The Synchronization Problem

When multiple threads share data, they can step on each other's toes. This is the single most important — and most dangerous — concept in concurrency.

6.1 The Race Condition

```
class Counter {
    int count = 0;

    void increment() {
        count++; // looks like one step but is actually THREE:
                // 1. read count
                // 2. add 1
                // 3. write back
    }
}

public class Main {
    public static void main(String[] args) throws Exception {
        Counter c = new Counter();

        Thread t1 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) c.increment();
        });
        Thread t2 = new Thread(() -> {
            for (int i = 0; i < 1000; i++) c.increment();
        });

        t1.start();
        t2.start();
        t1.join();
        t2.join();

        System.out.println(c.count); // Expected: 2000. Reality: usually
less!
    }
}
```

What Goes Wrong

Thread 1	Thread 2
read count = 5	read count = 5
add 1 = 6	add 1 = 6
write 6	write 6
v	v

Both threads thought count was 5, both wrote back 6.
Final result: 6, not 7. One increment was LOST.

This is a RACE CONDITION. Two threads racing to update the same data, and one wins while the other's work disappears. The result depends on TIMING — making bugs hard to reproduce.

7. synchronized — The Built-in Lock

Java's synchronized keyword makes a section of code MUTUALLY EXCLUSIVE — only one thread can be inside it at a time. Other threads wait their turn.

7.1 Synchronized Method

```
class Counter {
    int count = 0;

    public synchronized void increment() {
        count++;    // safe – only one thread at a time
    }
}
```

Adding synchronized to a method means: only one thread can be executing this method on a given instance at a time. Other threads block (wait) until the lock is released.

7.2 Synchronized Block

You don't always want to lock the entire method. You can lock just a section:

```
class Counter {
    int count = 0;
    private final Object lock = new Object();

    public void increment() {
        // non-critical setup here

        synchronized (lock) {    // only this section is locked
            count++;
        }
    }
}
```

```
        // non-critical work here
    }
}
```

Best practice — use a dedicated lock object: Locking on a dedicated 'private final Object lock' (instead of 'this' or a class) prevents anyone outside your class from accidentally locking the same object and causing deadlocks. It's a small step that prevents big bugs.

7.3 The Cost of synchronized

Synchronization has overhead — threads have to coordinate, which is slower than non-synchronized code. Lock only what you must, for as little code as necessary. Over-locking kills the speedup you wanted from threading.

8. Atomic Variables — A Lighter Alternative

For simple counters and flags, the `java.util.concurrent.atomic` package provides classes that handle synchronization internally — without explicit synchronized blocks.

```
import java.util.concurrent.atomic.AtomicInteger;

class Counter {
    AtomicInteger count = new AtomicInteger(0);

    public void increment() {
        count.incrementAndGet();    // atomic and thread-safe
    }

    public int get() {
        return count.get();
    }
}
```

`AtomicInteger` uses low-level CPU instructions (compare-and-swap) to update the value atomically — much faster than acquiring and releasing a synchronized lock. For simple counters and flags, prefer Atomic classes.

Class	Purpose
AtomicInteger	Thread-safe int counter
AtomicLong	Thread-safe long counter
AtomicBoolean	Thread-safe boolean flag
AtomicReference<T>	Thread-safe object reference

9. ExecutorService — Thread Pools (The Modern Way)

Creating a new Thread for every task is expensive. For most real-world work, you should use a THREAD POOL — a pre-created group of threads that pick up tasks from a queue.

Basic Usage

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) {
        ExecutorService executor = Executors.newFixedThreadPool(4);

        for (int i = 1; i <= 10; i++) {
            int taskNum = i;
            executor.submit(() -> {
                System.out.println("Task " + taskNum +
                    " on " + Thread.currentThread().getName());
            });
        }

        executor.shutdown();    // important – release threads when done
    }
}
```

4 threads handle 10 tasks. Each thread grabs a task, finishes it, then grabs another — no thread creation overhead per task.

Types of Thread Pools

Factory Method	Use Case
newFixedThreadPool(n)	Fixed number of threads (predictable workload)
newCachedThreadPool()	Creates threads as needed, reuses idle ones
newSingleThreadExecutor()	One thread — tasks run in order, one at a time
newScheduledThreadPool(n)	For scheduled / repeated tasks (like cron)

9.1 Callable — Tasks That Return a Result

Runnable doesn't return anything. Callable<T> does — and it can throw checked exceptions too.

```
import java.util.concurrent.*;
```

```

ExecutorService executor = Executors.newFixedThreadPool(2);

Future<Integer> result = executor.submit(() -> {
    Thread.sleep(2000);
    return 42;
});

System.out.println("Doing other work...");
Integer value = result.get(); // blocks until the result is ready
System.out.println("Got: " + value); // 42

executor.shutdown();

```

Future<T> is the placeholder for the value that's being computed. Call get() to wait for it (blocking) or check isDone() to see if it's ready.

10. Concurrent Collections

Regular collections like ArrayList and HashMap are NOT thread-safe. Modifying them from multiple threads causes ConcurrentModificationException, lost updates, or worse. For shared data, use the concurrent versions:

Collection	Use Case
ConcurrentHashMap	Thread-safe HashMap
CopyOnWriteArrayList	Thread-safe ArrayList (best for read-heavy workloads)
BlockingQueue<T>	Thread-safe queue with put() / take() that block
ConcurrentLinkedQueue<T>	Non-blocking thread-safe queue

```

import java.util.concurrent.ConcurrentHashMap;

ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();
map.put("Aman", 92);
map.computeIfAbsent("Riya", k -> 0);
map.merge("Aman", 1, Integer::sum); // atomic increment

// Safe to read AND write from multiple threads

```

11. Deadlock — A Classic Threading Bug

A DEADLOCK happens when two threads are each holding a lock the other needs, and neither will release theirs. Both wait forever.

Thread A	Thread B

holds Lock 1		holds Lock 2
wants Lock 2		wants Lock 1
	← waits forever →	
v		v
D E A D L O C K		

Example

```
Object lock1 = new Object();
Object lock2 = new Object();

Thread t1 = new Thread(() -> {
    synchronized (lock1) {
        Thread.sleep(100);
        synchronized (lock2) { ... }    // waits for lock2
    }
});

Thread t2 = new Thread(() -> {
    synchronized (lock2) {
        Thread.sleep(100);
        synchronized (lock1) { ... }    // waits for lock1
    }
});
```

How to avoid deadlocks: Always acquire locks in the SAME ORDER across all threads. If Thread A locks (1, then 2), Thread B should also lock (1, then 2). Never (2, then 1). This simple rule prevents most deadlocks. Also: use `tryLock()` with timeouts for finer control.

12. Common Mistakes

Mistake 1: Calling `run()` instead of `start()`

```
t.run();        // x runs on the SAME thread – no parallelism
t.start();      // ✓ starts a NEW thread
```

Mistake 2: Forgetting to synchronize shared data

If two threads modify the same variable without synchronization, you'll get random, hard-to-reproduce bugs. Synchronize, use atomic classes, or avoid sharing.

Mistake 3: Not shutting down `ExecutorService`

```
ExecutorService executor = Executors.newFixedThreadPool(4);
// ... submit tasks ...
executor.shutdown();    // MUST call this, or your program won't exit
```

Mistake 4: Catching InterruptedException silently

```
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    // BAD – swallows the interrupt signal silently
}

// GOOD – restore the interrupted flag
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
```

Mistake 5: Using non-thread-safe collections from threads

```
HashMap<String, Integer> shared = new HashMap<>(); // x not thread-safe
ConcurrentHashMap<String, Integer> shared = new ConcurrentHashMap<>(); //
✓
```

Mistake 6: Locking on 'this' carelessly

synchronized(this) is fine for simple cases — but if external code locks on your object, you might get unexpected blocking. Use a private final Object lock for serious code.

Mistake 7: Premature parallelization

Don't reach for threads on every problem. Single-threaded code is simpler, easier to debug, and often fast enough. Profile FIRST — only parallelize when there's a clear bottleneck.

13. Best Practices

- Prefer ExecutorService over raw Thread for production code
- Use lambda Runnables for short tasks — much cleaner than Thread subclasses
- Synchronize as little code as possible — locks have a cost
- Use Atomic classes for simple counters/flags instead of synchronized
- Use ConcurrentHashMap and other concurrent collections for shared data
- Lock objects in a CONSISTENT ORDER across threads to prevent deadlocks
- Always restore the interrupted flag when catching InterruptedException
- Shut down ExecutorService when you're done with it — otherwise the JVM hangs
- Make objects IMMUTABLE when possible — immutable objects are automatically thread-safe

14. Quick Reference

Concept	Meaning
Thread	The class representing a thread of execution
Runnable	Functional interface — <code>run()</code> with no return value
Callable<T>	Like <code>Runnable</code> but returns <code>T</code> and can throw
start()	Start a new thread and call <code>run()</code> in it
run()	The method that contains the thread's work
sleep(ms)	Pause current thread for <code>ms</code> milliseconds
join()	Wait for another thread to finish
interrupt()	Politely request a thread to stop
setDaemon(true)	Mark as background thread (doesn't block JVM exit)
synchronized	Lock a method or block — one thread at a time
volatile	Force fresh reads of a field across threads
AtomicInteger	Thread-safe <code>int</code> counter
ConcurrentHashMap	Thread-safe <code>HashMap</code>
ExecutorService	Manages a pool of threads
Future<T>	Placeholder for a future result
shutdown()	Tell <code>ExecutorService</code> no more tasks coming

15. Practice Problems

Try each one — they cover the patterns you'll see in real concurrent code.

1. Create 3 threads using lambdas, each printing its name 5 times. Observe interleaved output.
2. Write a thread that prints "tick" once per second for 10 seconds using `Thread.sleep`.
3. Create two threads that increment a shared counter 10,000 times each. Observe the race condition (`sum != 20,000`).
4. Fix the race condition above using `synchronized`.
5. Fix the same race condition using `AtomicInteger` instead.
6. Use `Thread.join()` to wait for 3 worker threads to finish before printing "All done."
7. Create an `ExecutorService` with 4 threads. Submit 20 tasks that each sleep 1 second.

8. Use Callable + Future to compute the factorial of 10 in a background thread.
9. Build a simple producer-consumer using BlockingQueue (one thread adds, another consumes).
10. Use ConcurrentHashMap with computeIfAbsent to safely build a word frequency map from multiple threads.
11. Create a daemon thread that logs heartbeat "alive" every 500ms. Show that the JVM exits when main does.
12. Demonstrate a deadlock: two threads each holding one lock and trying to acquire the other. Then fix it by locking in the same order.
13. Use newScheduledThreadPool to schedule a task that runs every 2 seconds for a minute.

16. Final Takeaway

Multithreading is what makes a Java backend handle thousands of users at once, an Android app stay smooth while downloading, and a data processor finish in seconds instead of minutes. It's also one of the topics that humbles even experienced developers — concurrency bugs are subtle, intermittent, and hard to debug.

The good news: 90% of real-world threading needs are covered by ExecutorService + lambda Runnables + AtomicInteger + ConcurrentHashMap. Master those four tools and you'll be ahead of most developers.

Key Takeaway: Use ExecutorService instead of raw Thread. Use AtomicInteger/ConcurrentHashMap instead of synchronized when you can. Always shut down executors. Always restore the interrupted flag. Avoid sharing mutable state when possible — immutable objects are automatically thread-safe. Get these habits right and you avoid the worst threading bugs in real code.

17. What's Next?

Now that threading basics are in place, the next topics to explore are:

- CompletableFuture — modern async programming with chaining
- Locks API — ReentrantLock, ReadWriteLock for finer control
- Semaphore and CountdownLatch — for advanced coordination
- Parallel streams in depth — when they help vs hurt
- ForkJoinPool — recursive parallel work splitting
- Reactive programming basics — Project Reactor, RxJava
- Date and Time API — java.time, LocalDate, Instant, Duration